VIT SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

MODERN AIRLINE LOGISTICS & SUSTAINABILITY MANAGEMENT SYSTEM

Database project for BACSE202 Lab Component

Pranali Paygonda Patil

25BCE2957, B.Tech Computer Science and Engineering

School of Computer Science and Engineering, AY 2026–27

AIRLINE OPERATIONS CONNECTED THROUGH DATA

Flight, fleet, crew, passenger, cargo and emissions

Abstract

A centralized relational system for connected airline operations

The project brings essential airline logistics records into one structured database. It links aircraft and flights with crew assignments, passenger loyalty, specialized cargo, emissions, and disruption events.

Operational control

Search and maintain linked records through a browser console.

Sustainability visibility

Capture fuel and CO_2 at flight level for later analysis.

Decision support

Use joins, aggregates, grouping, and subqueries to answer operational questions.

9 relational entities

Oracle + FastAPI web app

Cold-chain + emissions focus

The operating problem

Airline logistics crosses teams, systems, and time-sensitive decisions

1

Schedules

Flight and aircraft status can drift across separate records.

2

People

Crew assignments must stay connected to flights and duty details.

3

Cargo

Temperature-sensitive goods need handling requirements and checkpoints.

4

Disruptions

Causes and delay duration need a consistent event history.

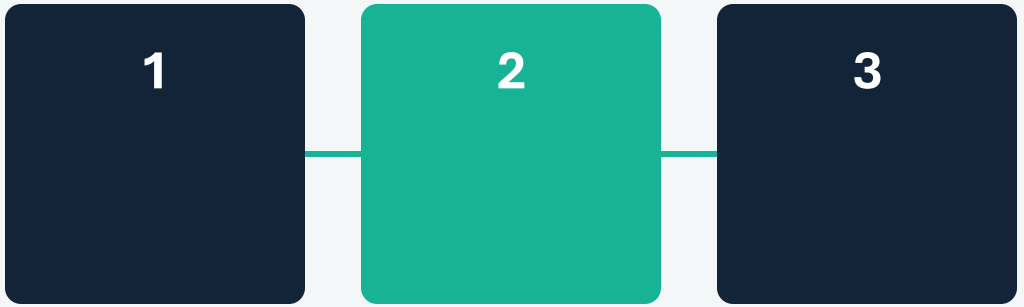
Project response

A shared relational model links operational events to the flight that anchors them.

Why sustainability belongs in logistics

The project was inspired by the need to make environmental impact visible alongside operations

Every flight already creates operational records. Adding fuel and CO_2 fields makes sustainability measurable at the same unit of work: the flight.



Record

Fuel liters + CO_2 tons

Relate

Tie each log to a flight

Analyse

Compare routes and aircraft

Inspiration: sustainability through measurement and operational awareness

Project objectives

What the database and interface are designed to support

01	**Integrate**	Relate operational records through keys and constraints.
02	**Protect quality**	Reduce inconsistent references and invalid values.
03	**Monitor**	Keep cold-chain readings, emissions, and disruptions queryable.
04	**Inform**	Use SQL summaries and joins to support operational decisions.

Users and business stakeholders

A coursework prototype for roles that depend on a reliable shared view

Operations controller	Review departures, statuses, disruptions, and linked resources.
Cargo coordinator	Check category, shipment weight, required temperature, and current reading.
Sustainability analyst	Summarize fuel and CO_2 logs by flight or route.
Database administrator	Maintain schema, accounts, constraints, and database availability.

Current prototype: all registered users have the same access. Separate job roles and permissions remain future work.

MODERN AIRLINE LOGISTICS

Functional scope

The report defines nine entities; the current browser interface exposes operational views

DATABASE DOMAIN

- Aircraft and flight scheduling
- Crew and flight assignments
- Passenger loyalty and baggage

OPERATIONAL USE

- Specialized cargo and temperature checks
- Fuel and CO_2 logs
- Disruption events and response status

The FLIGHT_CREW relationship exists in the schema; the current UI does not provide a dedicated assignment screen.

Solution architecture

Browser requests flow through a local API to Oracle XE



Oracle changes appear in the UI after refresh; the page polls every five seconds.

localhost:8001 | Signed-in routes | Oracle commit on writes

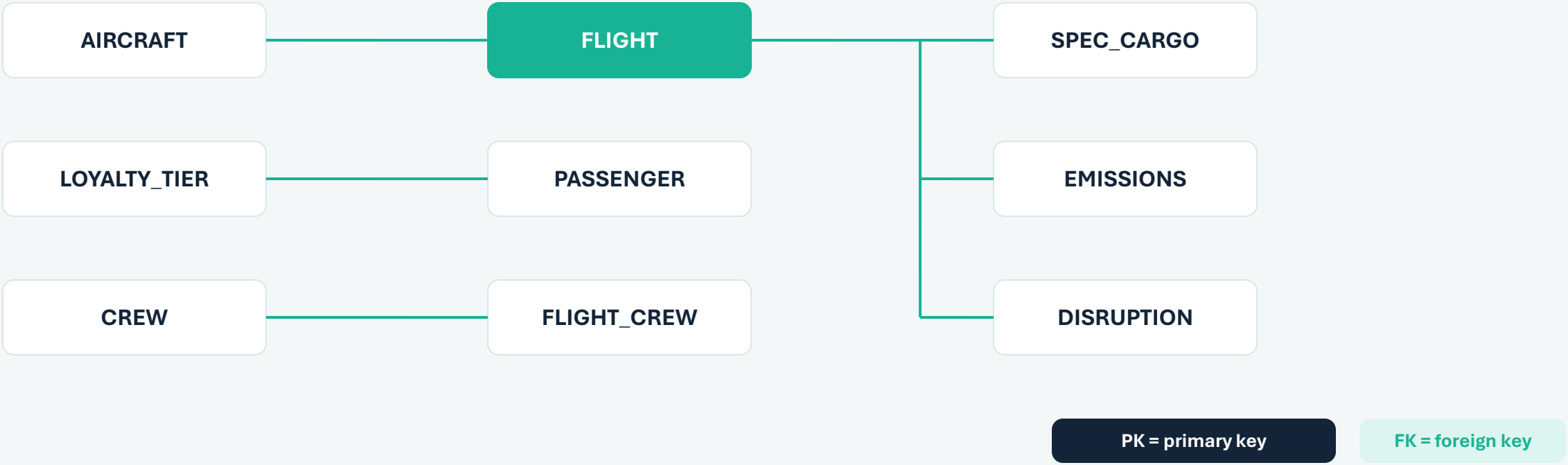
Technology and practical implementation

The runnable codebase reflects the current implementation

FRONTEND	**Single-page HTML/CSS/JavaScript console served by FastAPI**
API	Python FastAPI with JSON endpoints and CRUD routes
DATABASE	Oracle XE via python-oracledb Thin mode
CONFIGURATION	Local .env variables for Oracle user, password, and DSN
RUN PATH	Start Uvicorn, then open http://127.0.0.1:8001

Entity relationship model

Nine entities organize the airline domain around FLIGHT



FLIGHT_CREW resolves the many-to-many flight and crew relationship.

Relational schema at a glance

Primary keys identify records; foreign keys preserve cross-table links

CORE

AIRCRAFT (Aircraft_ID PK)

FLIGHT (Flight_ID PK, Aircraft_ID FK)

PEOPLE

CREW (Crew_ID PK)

FLIGHT_CREW (Assignment_ID PK, Flight_ID FK, Crew_ID FK)

LOYALTY_TIER (Tier_ID PK)

PASSENGER (Passenger_ID PK, Tier_ID FK)

OPERATIONS

SPEC_CARGO (Cargo_ID PK, Flight_ID FK)

EMISSIONS (Log_ID PK, Flight_ID FK)

DISRUPTION (Event_ID PK, Flight_ID FK)

Foreign-key paths

FLIGHT references AIRCRAFT. FLIGHT_CREW links flights with crew. PASSENGER references LOYALTY_TIER. SPEC_CARGO, EMISSIONS and DISRUPTION records reference FLIGHT.

Relationships and cardinality

The model links each operational record to its parent entity

AIRCRAFT	1 : many	FLIGHT
FLIGHT	many : many via FLIGHT_CREW	CREW
FLIGHT	1 : many	SPEC_CARGO / EMISSIONS / DISRUPTION
LOYALTY_TIER	1 : many	PASSENGER

The associative table stores one row per crew-to-flight assignment, including shift hours.

Data integrity and normalization

Constraints make the database enforce rules at the point of change

PRIMARY KEY	Unique identity for each entity row
FOREIGN KEY	Valid links between related records
UNIQUE	No duplicate tier names or passport numbers
NOT NULL / CHECK	Required fields and allowed-value rules
DEFAULT	A defined value when an insert omits one

Logical separation reduces repeated facts while referential integrity keeps records connected.

MODERN AIRLINE LOGISTICS

Flight operations workflow

A flight record anchors connected planning and monitoring data

01	02	03	04	05
Select aircraft	**Schedule route**	**Assign crew**	**Load cargo**	**Track outcomes**
FLIGHT.AIRCRAFT_ID	Route, departure and arrival	FLIGHT_CREW links people	Shipment, category and temperature	Disruption and emissions logs

Operational rule

A flight must reference a real aircraft; child events must reference a valid flight.

MODERN AIRLINE LOGISTICS

Cold-chain cargo management

Temperature-sensitive shipments carry their own monitoring fields

CATEGORY

Medical, perishable, hazardous and standard

REQUIRED

Shipment target temperature (°C)

CURRENT

Latest recorded temperature (°C)

TRACKING

Status label for shipment follow-up

Current limitation

The prototype stores readings and status; automated sensor feeds and alert thresholds are future work.

Passenger loyalty and baggage

A tier reference keeps allowance rules separate from passenger records

SUMMIT	+15 kg
AURORA	+5 kg
HORIZON	+0 kg

PASSENGER references LOYALTY_TIER — Tier name and extra baggage allowance remain in the reference table.

Disruption management

Each disruption record captures the cause, delay, time, and response state

- **Weather**
- **Technical issue**
- **Air traffic control**
- **Crew availability**
- **Security / ground handling**

EVENT RECORD FIELDS

- Delay duration in minutes
- Reason code
- Event date and time
- Response status

Management can group events by reason or status to see recurring pressure points.

MODERN AIRLINE LOGISTICS

Flight-level sustainability records

The EMISSIONS table stores fuel and CO_2 measurements against each flight

FUEL_LITERS

Fuel consumed

CO2_TONS

Carbon dioxide recorded

LOGGED_AT

Time of measurement

FLIGHT_ID

Operational link

Illustrative front-end sample logs total 84,000 L of fuel and 223.2 t of CO_2.

Use live Oracle data for operational reporting; these bundled values are demonstrative.

SQL portfolio in Review 0

The coursework demonstrates both record maintenance and analytical queries

QUERY TOOLKIT

- DDL: create tables with constraints
- DML: insert, update, and delete
- SELECT and single-row functions
- Aggregate functions and GROUP BY

ANALYTICS PATH

- HAVING for group-level filters
- Subqueries for nested conditions
- JOINs across operational entities
- Five initial rows per table in the report

Examples in the report cover aircraft and flight, passenger and tier, crew and flight, cargo and flight, and disruption and flight.

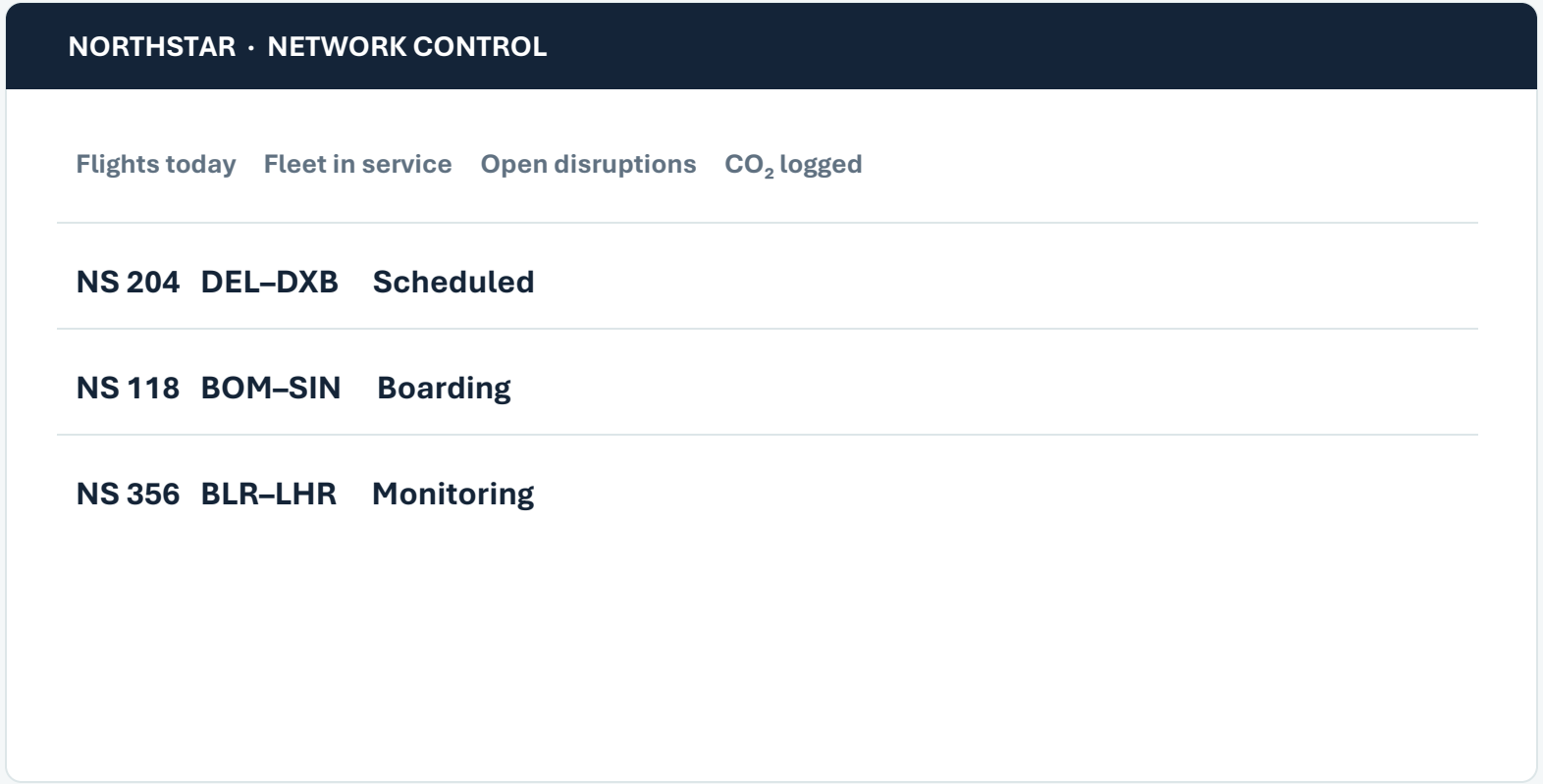
SQL and business decisions

The relational model supports practical questions without claiming automated optimization

FLEET	**Which aircraft operate which routes?**	Review aircraft assignment and reported fuel metrics.
COLD CHAIN	**Which shipment readings differ from targets?**	Prioritize manual inspection or follow-up.
DISRUPTIONS	**Which reasons recur with longer delays?**	Focus investigation on repeated causes.
SUSTAINABILITY	**How do fuel and CO_2 vary by route?**	Compare operational records and identify candidates for review.

Browser operations console

A single interface brings the database entities into browsable views



Interface capabilities

- Entity navigation
- Search and status filters
- CSV export
- Record detail + CRUD forms

Sample values shown here come from the bundled front-end seed data.

API and data lifecycle

The current API reads Oracle data and commits create, update, and delete actions

READ	**GET /api/data**	Page reads Oracle rows
CREATE	**POST /api/{entity}**	Form values create a record
UPDATE	**PUT /api/{entity}/{id}**	Changed fields update a record
DELETE	**DELETE /api/{entity}/{id}**	Selected record is deleted

Oracle constraints remain authoritative; key generation must be configured for add forms where IDs are omitted.

Authentication and security design

The local prototype adds account sign-in around data routes

PASSWORD STORAGE

scrypt hash with random salt

SESSION

Signed HttpOnly cookie, 12-hour expiry

DATABASE CONFIG

Credentials live in local .env

ROUTE ACCESS

Operational data and CRUD require sign-in

Deployment boundary The README intends localhost coursework use. Keep the API off untrusted networks. The app does not implement role-based permissions.

Live demonstration walkthrough

A short review path that shows the application end to end

1	**Start**	Launch the local FastAPI service and confirm Oracle health.
2	**Sign in**	Create or use a local account.
3	**Explore**	Open flights, fleet, cargo, disruptions, emissions.
4	**Change**	Add or edit a record and observe the refreshed view.
5	**Analyse**	Show a SQL join or grouped summary in the database client.

Project demo video

The Google Drive link

GOOGLE DRIVE DEMO LINK

https://drive.google.com/drive/folders/1iJCeO4C2KEm1-Avz9S9sdOv1OaXjlvBR?usp=sharing

Practical setup and deployment

The README documents a local development workflow

01 Install Python 3.10+ and requirements

02 Copy .env.example to .env and set Oracle credentials + DSN

03 **Start Uvicorn on 127.0.0.1:8001**

04 Open the local URL in a browser

05 Register, then use the console

Oracle tables and service must match the configured schema. XEPDB1 differs from the README's CDB$ROOT setup.

Verification and review evidence

The report documents SQL exercises; the code documents app behavior and setup

DATABASE REVIEW

Create tables with constraints
Insert sample rows
Update and delete demonstrations
Join, aggregate, group, HAVING, subquery

APPLICATION REVIEW

Health endpoint for Oracle connectivity
Signed-in data routes
CRUD requests to mapped tables
Five-second refresh from Oracle

Important evidence limit

The browser’s chart and bundled records are illustrative. Use live Oracle rows in the demo for operational results.

Business value and operating model

Potential value comes from clearer coordination and more usable operational records

Fewer handoffs	Shared flight-linked records reduce lookup across separate lists.
Cargo accountability	Temperature requirements and readings can be reviewed together.
Environmental reporting	Fuel and CO_2 logs create a basis for route-level comparison.
Operational learning	Disruption history can reveal recurring causes and delay patterns.

Adoption path **Pilot with operations and cargo teams, validate data quality, then add reporting and access controls.**

Future scope

A staged roadmap can grow the coursework prototype into a more operational tool

NEAR TERM	**Role-based access, stronger validation, explicit ID strategy**
NEXT	Live temperature sensor feeds, threshold alerts, audit trail
LATER	Emissions dashboards, route benchmarks, exportable reports
SCALE	Cloud deployment with monitoring, backup and recovery, and integrations

All roadmap items are proposals; they are not implemented in the current code.

THANK YOU

MODERN AIRLINE LOGISTICS & SUSTAINABILITY MANAGEMENT SYSTEM

Project Work and Future Enhancement

Modern Airline Logistics and Sustainability Management System

Project purpose

This project develops a database-backed system for organizing airline logistics in one connected model. Airline work depends on accurate links among aircraft, flights, crew, passengers, cargo and operational events. When those records sit in separate lists, staff may face inconsistent information, slower coordination and limited visibility into delays or environmental impact. The project combines a relational database with a browser console so users can review and maintain key records in a shared operational view.

Project objectives

- Integrate flight, aircraft, crew and passenger records in a relational model.
- Represent cold-chain requirements, flight-level fuel and CO_2, and delay events.
- Use keys and constraints to protect relationships and required data.
- Demonstrate how a shared database can support search, record maintenance and reporting.

Work completed

The Review 0 work defines a normalized relational design with nine entities: AIRCRAFT, FLIGHT, CREW, FLIGHT_CREW, LOYALTY_TIER, PASSENGER, SPEC_CARGO, EMISSIONS and DISRUPTION. FLIGHT acts as the operational anchor. Primary and foreign keys preserve record identity and relationships. UNIQUE, NOT NULL, CHECK and DEFAULT constraints help prevent invalid or duplicate data. The many-to-many relationship between flights and crew is represented through FLIGHT_CREW, which also records shift hours.

The database coursework demonstrates table creation, inserts, updates and deletions, along with SELECT statements, single-row and aggregate functions, GROUP BY, HAVING, subqueries and joins. These operations support both routine record management and questions that combine information across related tables.

Relational design in practice

Each flight references one aircraft, and crew members connect to flights through FLIGHT_CREW. A loyalty tier can apply to many passengers, while each passenger references a tier. Cargo, emissions and disruption records point to the flight they describe. These relationships support queries across scheduling, staffing, baggage, cargo handling, environmental logging and delay history without repeating core flight details in every related record.

Sustainability motivation

The sustainability focus comes from making environmental impact visible alongside flight operations. EMISSIONS stores fuel consumed, CO_2 recorded and the time of each log, linked to a flight. This creates a basis for later route or aircraft comparisons. The project records and organizes sustainability data; it does not claim that the prototype has reduced emissions.

Implementation and practical use

The current application serves a single-page HTML, CSS and JavaScript interface through a local FastAPI service. Python's python-oracledb driver connects to Oracle XE in Thin mode. The browser requests JSON data from the API, and create, update and delete actions are committed to Oracle. The README describes running the service with Uvicorn at 127.0.0.1:8001 after the local Oracle credentials and service name are configured.

Operational features

- The console provides views for flights, aircraft, crew, passengers, specialized cargo, emissions and disruptions. Loyalty tiers serve as a passenger lookup.
- Users can search records, filter by status, export CSV files, inspect record details and use forms to add or edit records. The interface refreshes Oracle data every five seconds.
- Cargo records include category, weight, required temperature, current temperature and tracking status. Disruption records capture a flight, reason, delay duration, event time and status.
- Emission records link fuel and CO_2 values to a flight. SQL joins and grouped summaries can use these links to compare routes or investigate patterns.
- Account registration and sign-in protect the operational API. Passwords use scrypt hashes, and sessions use a signed HttpOnly cookie with a 12-hour expiry.

Business and operational relevance

A shared flight-linked record can help operations staff review aircraft assignments and disruption status without reconciling separate lists. Cargo coordinators can compare a shipment's required and current temperature. Sustainability analysts can prepare route-level comparisons from fuel and CO_2 logs. Managers can group disruption events by reason or status to identify recurring causes for further investigation. These are potential uses of the data model, not measured improvements from a live airline deployment.

Current scope and limitations

The Review 0 document lists MySQL 8.0, while the current runnable code and README use Oracle XE. The database design describes FLIGHT_CREW, but the browser application does not provide a dedicated screen for managing crew-to-flight assignments. User accounts currently share the same access level, the README limits use to localhost coursework, and Oracle key generation must be configured for add forms when an ID is omitted. Bundled browser sample records and the overview chart are illustrative; use live Oracle data for operational reporting.

Future enhancements

The next stage should improve data quality and operational usefulness before expanding deployment. Enhancements below are proposals and are not part of the current implementation.

1. Strengthen validation and access control

Add role-based permissions for operations, cargo and sustainability users. Validate date order, numeric ranges, status values and required fields in both the interface and API. Define an explicit identity strategy for every table and retain an audit history of who changed each record and when. These changes would make the system safer to use and easier to review.

2. Add live cold-chain monitoring

Extend cargo records to hold timestamped sensor readings rather than a single current temperature. Let staff configure acceptable ranges by cargo type, raise an alert when a reading crosses a limit, and record acknowledgement and resolution. A pilot could begin with one shipment category and compare sensor readings with the existing manual workflow.

3. Build sustainability analysis

Create dashboards for fuel and CO_2 by flight, route and aircraft. Record the source and method used for each emissions value, and add route distance or payload information before calculating intensity measures. This would make comparisons more meaningful and help analysts distinguish a higher total caused by a longer route from a less efficient operation.

4. Improve disruption and fleet planning

Add summaries for delay duration and disruption reason, then connect those findings with schedule and aircraft data. Later versions could incorporate weather or air-traffic feeds to help staff see developing risks. Any recommendation feature should explain which records support it and leave operational decisions with authorized staff.

5. Prepare for reliable deployment

Before use beyond a local demonstration, add protected secret storage, encrypted network traffic, structured logs, monitoring, backups and recovery procedures. Test database constraints and API behavior with representative cases, document setup for a clean environment, and integrate other systems only through defined and access-controlled interfaces.

Conclusion

The project establishes a practical foundation for connected airline logistics records and introduces sustainability and cold-chain data into the operational model. Its strongest next steps are reliable validation, traceable live cargo readings and clearly sourced emissions analysis. With these improvements, the prototype could provide more dependable information for airline coordination while preserving a clear record of how each operational decision is supported.